République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université M'hamed Bougara-Boumerdès

Domaine : Mathématiques et Informatique
Filière : Informatique
Spécialité : Intelligence artificielle appliquée
Module : Web Avancé et Micro-Services & Apprentissage automatique

Rapport :

Système Intelligent de Traduction de la Langue des Signes basé sur Microservices et Deep Learning

Présenté par:

Mlle.BELLIL Ikram
Mlle. MERABIA Meriem
Mlle. hayet bendjakboub

Encadrant:

M. TOUAZI
M.GACEB
Mme.BOUGHANEM

I. Introduction

Dans un monde de plus en plus numérique, l'accessibilité des technologies constitue un enjeu majeur pour garantir l'inclusion de toutes les catégories de la société. Parmi les populations concernées, **les personnes sourdes et malentendantes** rencontrent encore aujourd'hui de nombreuses **difficultés dans leur communication** quotidienne, notamment **avec les personnes ne maîtrisant pas la langue des signes**. Cette barrière de communication limite leur accès à l'information, à l'éducation et aux services numériques.

Avec les avancées de l'intelligence artificielle, en particulier **le Deep Learning** et la **vision par ordinateur**, il est désormais possible de reconnaître automatiquement des **gestes et de les traduire en texte**. Parallèlement, **l'architecture microservices** permet de concevoir **des applications modernes, flexibles et scalables, capables d'intégrer** des composants complexes comme les modèles d'IA.

Dans ce contexte, ce projet propose la conception **d'un système intelligent** permettant de **traduire la langue des signes en temps réel**, en s'**appuyant** sur une **architecture microservices** et **un modèle de Deep Learning performant**.

Problématique :

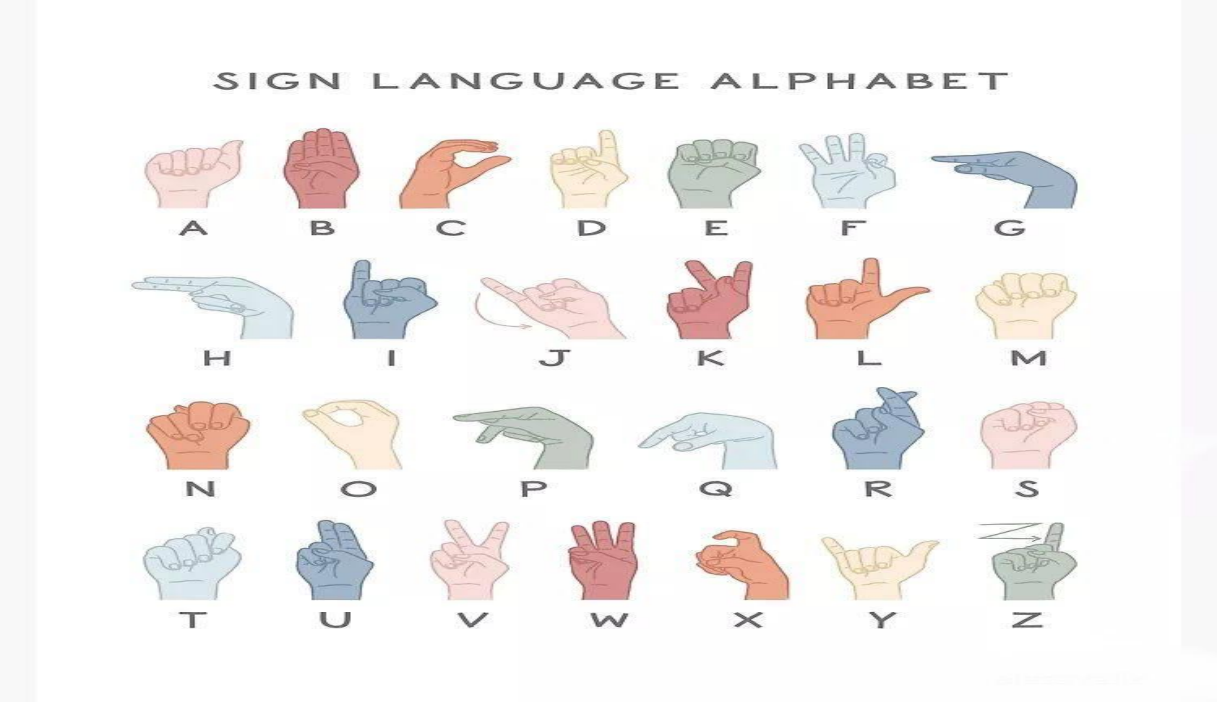
> ***Comment concevoir un système intelligent capable de traduire la langue des signes en temps réel, en combinant une architecture microservices scalable et un modèle de deep learning performant ?***

II. projet Description du système

Le projet consiste à concevoir et développer un système intelligent de traduction de la langue des signes en texte, basé sur une **architecture microservices** et des techniques de Deep Learning. **L'objectif** principal est de **faciliter la communication entre les personnes sourdes et malentendantes et les personnes ne maîtrisant pas la langue des signes.**

Le système fonctionne selon un pipeline complet. Tout d'abord, **l'utilisateur accède** à une interface web via laquelle il peut **activer sa caméra**. Les **gestes** de la main sont alors **capturés en temps réel** sous forme d'**images**. Ces images sont ensuite envoyées vers un **service d'intelligence artificielle** qui **analyse les gestes** à l'aide d'un modèle de **Deep Learning** entraîné. **Le modèle identifie le geste** et le traduit **en texte**. Enfin, **le résultat est affiché** directement sur l'interface utilisateur.

L'**architecture** du système repose sur plusieurs **microservices indépendants**, notamment un service d'**authentification** pour la gestion des utilisateurs, une **API métier** pour le traitement des requêtes, un service **d'intelligence artificielle** pour la reconnaissance des gestes, ainsi qu'un **système de communication asynchrone** pour le traitement. Cette organisation permet d'assurer une **meilleure scalabilité, une maintenance facilitée et une évolution indépendante** de chaque composant.



PARTIE 1 : ARCHITECTURE MICROSERVICES

1.1 Introduction de la partie

Pour répondre à la problématique posée, nous avons conçu une architecture basée sur **les microservices** afin de rendre le système **modulaire, scalable et facile** à maintenir. Cette approche permet de **décomposer l'application** en plusieurs services **indépendants**, chacun ayant un rôle bien défini.

Dans notre projet, cette **architecture est particulièrement** adaptée car **elle permet** de **séparer** les différentes **fonctionnalités du système**, notamment **l'authentification** des utilisateurs, **la gestion des requêtes** métier, ainsi que **le traitement** basé sur **l'intelligence artificielle**.

Ainsi, l'utilisation des microservices constitue **une base solide** pour construire un **système distribué performant**, capable de gérer des traitements **en temps réel** comme la traduction de la langue des signes.

1.2 Choix de l'architecture microservices

Dans le développement des applications modernes, deux principales approches architecturales sont souvent utilisées : l'architecture monolithique et l'architecture microservices. Le choix entre ces deux modèles dépend des besoins du système, de sa complexité et des objectifs de scalabilité et de maintenance.

Architecture monolithique vs microservices

Une architecture monolithique repose sur une **seule application** regroupant toutes les fonctionnalités (interface utilisateur, logique métier, accès aux données). Toutes les composantes sont **fortement couplées et déployées ensemble**. Cette approche est **simple à développer** au début, mais devient rapidement **difficile à maintenir** et à faire **évoluer** lorsque le projet grandit.

À l'inverse, **une architecture microservices** consiste à **découper l'application** en plusieurs **services indépendants**, chacun ayant **une responsabilité spécifique**. Chaque **microservice** peut être **développé, déployé et maintenu séparément**. Ces services **communiquent entre eux via des API** ou **des systèmes de messagerie**.

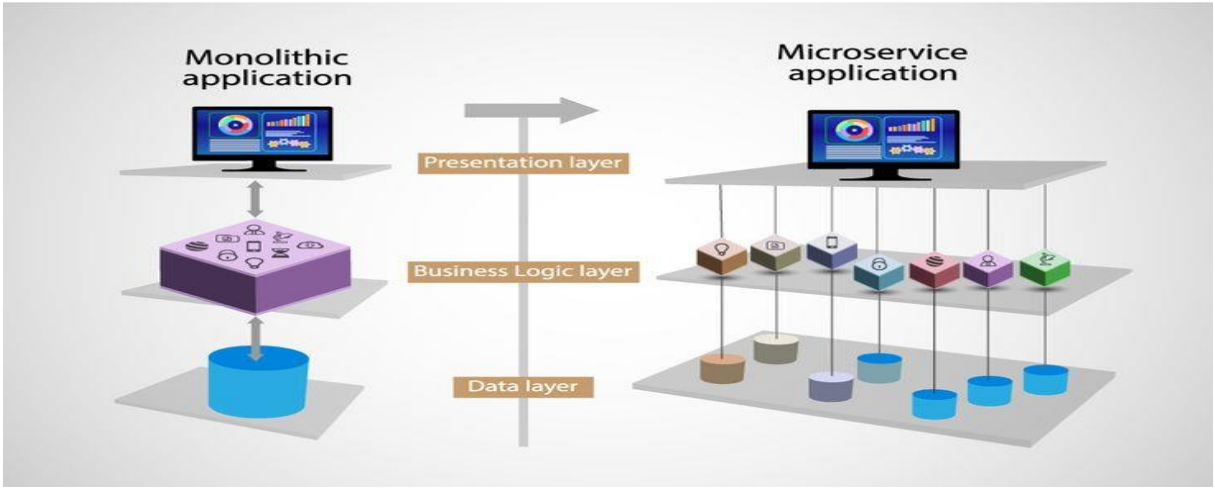
Avantages de l'architecture microservices

Scalabilité : L'architecture microservices permet de faire évoluer chaque service indépendamment selon les besoins. Par exemple, le service de reconnaissance des gestes peut être renforcé sans modifier les autres composants.

Maintenance facilitée : Chaque service étant indépendant, les mises à jour ou corrections peuvent être effectuées sans impacter l'ensemble du système. Cela réduit les risques de panne globale.

Indépendance des services : Chaque microservice possède sa propre logique, son propre cycle de vie et peut utiliser des technologies différentes. Cela permet une grande flexibilité dans le développement.

Cette architecture est donc particulièrement adaptée à notre projet de traduction de la langue des signes, car elle permet de **séparer les responsabilités** (authentification, API, IA, communication asynchrone) tout en **garantissant une meilleure performance globale.**



1.3 Architecture globale du système

- **expliquer chaque bloc :**

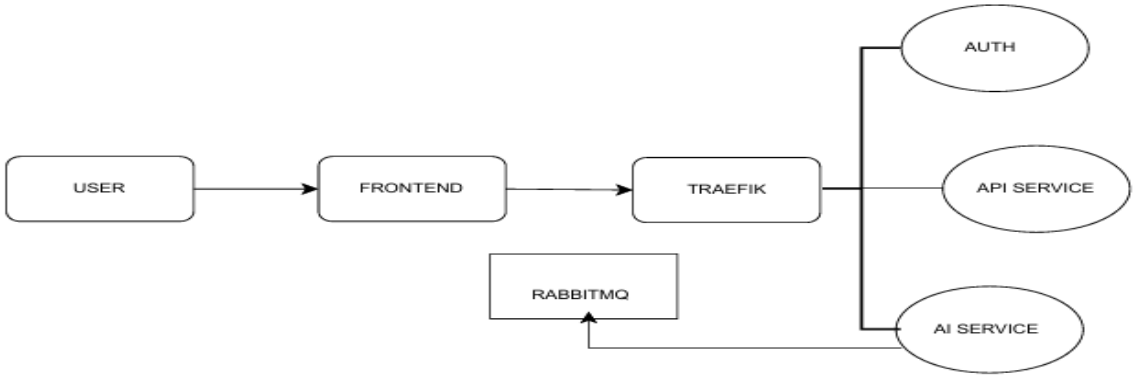
Le frontend constitue **le point d'entrée** de l'utilisateur. Il permet d'interagir avec le système via une interface web, notamment pour la capture des gestes en temps réel. Les requêtes sont ensuite redirigées par un reverse proxy **(Traefik),** qui joue le rôle de point d'entrée unique et de répartiteur de charge.

Les différentes fonctionnalités sont ensuite gérées par des microservices spécialisés :

- un service d'authentification pour la gestion des utilisateurs,
- un service API pour la logique métier,
- un service d'intelligence artificielle pour la reconnaissance des gestes,
- un système de communication asynchrone pour le traitement des tâches lourdes.

La communication entre ces services est assurée via des API REST et des messages asynchrones **(RabbitMQ),** garantissant ainsi une meilleure performance et une séparation claire des responsabilités.

- **schéma architecture**



1.4 Description détaillée des microservices

L'architecture proposée repose sur **une décomposition du système** en plusieurs **microservices indépendants**, chacun responsable **d'une fonctionnalité spécifique**. Cette organisation permet de **garantir une meilleure modularité, une évolutivité** accrue ainsi **qu'une maintenance simplifiée**. Les principaux services constituant le système sont décrits ci-dessous.

1.4.1 Service d'authentification (Auth Service)

Le service d'authentification constitue **un composant central du système**, **chargé de sécuriser** l'accès aux différentes fonctionnalités. **Il assure la gestion** des utilisateurs ainsi que **le contrôle des accès** aux ressources.

Son rôle principal consiste à **vérifier l'identité des utilisateurs** lors de **la connexion** et à générer des **tokens d'accès sécurisés**. Dans ce projet, un mécanisme basé sur les **JSON Web Tokens (JWT**) est utilisé. Ce type de token permet d'authentifier **les requêtes de manière stateless**, sans nécessiter de session côté serveur.

Un mécanisme de **contrôle d'accès** basé sur **les rôles (RBAC)** est mis en place. Il attribue à chaque **utilisateur un rôle** (administrateur ou utilisateur standard) avec des **permissions spécifiques.**

L'administrateur dispose **d'un accès complet**, tandis que **l'utilisateur standard** est **limité à certaines actions**. Cela améliore la sécurité et la gestion des accès.

Les principales fonctionnalités offertes par ce service incluent :

- l'authentification des utilisateurs via un **endpoint de connexion**,
- **la génération et la validation** des tokens **JWT,**
- la gestion des rôles et des **permissions (RBAC).**

Ce service expose généralement **des endpoints** tels que :

- **POST /login** : permet à l'utilisateur de s'authentifier et d'obtenir un token,
- **GET /verify** : permet de vérifier la validité d'un token,
- **GET /health** : utilisé pour vérifier l'état du service.

Ainsi, **ce microservice** garantit **la sécurité globale** du système en **contrôlant l'accès** aux **autres services**.

1.4.2 Service API métier (API Service)

Le service API représente **le cœur fonctionnel de l'application**. Il implémente la **logique métier** et **assure la gestion des données** manipulées par le système.

Ce service **reçoit les requêtes** provenant du **frontend (via le proxy)** et applique les **règles métier associées**. Il gère notamment **les opérations liées** aux **traductions**, à **l'historique** des interactions et aux **données utilisateurs**.

Ses principales responsabilités incluent :

- le traitement des requêtes liées à la traduction des gestes,
- la gestion des données (création, lecture, mise à jour),
- l'interaction avec la base de données,
- la communication avec les autres services.
- la validation des données entrantes afin de garantir leur cohérence et leur conformité,
- la gestion des erreurs pour assurer la fiabilité du système.

Parmi les endpoints exposés, on retrouve :

- **POST /interpretation-requests** : envoi d'une demande de traduction,
- **GET /translations** : récupération des résultats,
- **GET /health** : vérification de l'état du service

Ce service joue donc **un rôle central** en assurant **la cohérence** et **la gestion des flux de données** au sein du système.

Architecture interne du service API

Au niveau du **service API,** le projet est structuré selon les conventions du **framework Django REST Framework** afin d'assurer une organisation claire et maintenable du code. **Trois types de vues** ont été utilisés:

- **Function-Based Views (FBV)**
  Les Function-Based Views sont utilisées pour **les opérations simples** nécessitant un traitement direct et rapide.
  Dans ce projet, elles sont principalement utilisées pour :
  - la prédiction des gestes **(*predict*)**
  - la création de traductions **(*translation*)**

  Ce type de vue permet une implémentation simple et efficace pour les traitements légers.

- **Class-Based Views (CBV)**
  Les Class-Based Views sont utilisées pour les **fonctionnalités** nécessitant une meilleure **structuration et réutilisation** du code.
  Elles sont principalement utilisées pour :
  - la gestion de l'historique
  - les vues détaillées (***HistoryView, HistoryDetailView***)

  Cette approche permet une meilleure organisation et une maintenance facilitée du code.

- **ViewSets (Django REST Framework)**
  Les ViewSets sont utilisés pour générer automatiquement les endpoints REST, réduisant ainsi la quantité de code nécessaire tout en respectant les standards REST.
  Dans ce projet, on retrouve notamment :
  - **UserViewSet** : gestion des utilisateurs
  - **TranslationViewSet** : opérations CRUD sur les traductions
  - **HistoryViewSet** : gestion de l'historique

Cette organisation basée sur Function-Based Views, Class-Based Views et ViewSets permet d'optimiser la structure du service API en combinant simplicité, modularité et automatisation des endpoints REST.

1.4.3 Service d'intelligence artificielle (AI Service)

Le service d'intelligence artificielle est dédié à **la reconnaissance des gestes** de la langue des signes. Il constitue l'élément clé permettant de **transformer des données visuelles en texte interprétable**.

Ce service **reçoit des images** capturées par **la caméra** et **applique un modèle de Deep Learning** entraîné afin **d'identifier les gestes effectués**. Le modèle utilisé

repose **sur des réseaux de neurones convolutifs**, capables d'**extraire** automatiquement des **caractéristiques** pertinentes à **partir des images**.

Le fonctionnement du service peut être résumé comme suit :

- réception d'une image en entrée,
- prétraitement de l'image (redimensionnement, normalisation),
- prédiction du geste à l'aide du modèle,
- renvoi du résultat sous forme de texte.

Un endpoint typique de ce service est :

- **POST /predict** : reçoit une image et retourne la classe prédite.

Ce microservice est conçu pour être indépendant, ce qui permet de mettre à jour ou d'améliorer le modèle sans impacter les autres composants du système.

1.4.4 Service de traitement asynchrone (Worker Servic)

Le Worker Service est responsable de **l'exécution des tâches en arrière-plan**. Il repose sur **un système de file de messages**, généralement **implémenté à l'aide** de **RabbitMQ**, et utilise un framework tel que Celery pour gérer **les traitements asynchrones.**

Ce service permet de **déléguer les opérations coûteuses ou non critiques** afin de **ne pas bloquer les requêtes** principales. Il contribue ainsi à **améliorer les performances** globales du système.

Les tâches prises en charge par ce service peuvent inclure :

- l'enregistrement de l'historique des traductions,
- le traitement de données volumineuses,
- la gestion des logs et des notifications.

Le fonctionnement repose sur un modèle **producteur/consommateur** :

- les services principaux publient des messages dans une file,
- le worker consomme ces messages et exécute les tâches correspondantes.

Pipeline de traitement asynchrone :

- le service API envoie une tâche vers la file RabbitMQ,
- le Worker Service récupère la tâche depuis la file,
- le service d'intelligence artificielle traite l'image et effectue la prédiction,
- le résultat est renvoyé et exploité par le système (affichage ou stockage).

Cette approche permet **d'assurer une meilleure répartition** de la charge et **d'optimiser le temps de réponse**. et d'**améliorer la scalabilité** globale du système.

1.4.5 Frontend

Le frontend constitue **la couche d'interaction entre l'utilisateur et le système**. Il s'agit d'une application web développée (HTML,CSS JS), offrant une interface intuitive et accessible.

Ce composant permet à l'utilisateur de :

- s'authentifier,
- activer la caméra pour capturer les gestes,
- visualiser les résultats de traduction en temps réel.

Le frontend communique avec **les services backend** via des requêtes **HTTP (API REST)**, en incluant les **tokens JWT** pour sécuriser les échanges, chaque requête inclut un token JWT dans l'en-tête **Authorization**, permettant de vérifier l'identité de l'utilisateur et de contrôler l'accès aux ressources.
Cette approche garantit une communication sécurisée entre le client et les différents microservices.

L'interface est conçue pour être simple et ergonomique, afin de garantir une expérience utilisateur fluide, notamment pour les personnes sourdes ou malentendantes.

1.5 Communication entre services

Dans **une architecture microservices**, **la communication** entre les différents composants **constitue un élément essentiel** pour assurer le bon fonctionnement du système. Dans ce projet, **deux modes de communication** complémentaires ont été adoptés : une communication **synchrone basée sur les API REST** et une communication **asynchrone reposant sur un système de messagerie.**

1.5.1 Communication synchrone : API REST

La **communication synchrone** est assurée via des **API REST**, permettant aux **différents services d'échanger des données en temps réel**. Ce mode de communication est principalement **utilisé** pour **les interactions nécessitant une réponse immédiate**, notamment entre **le frontend et les services backend**.

Dans ce système, le frontend envoie des **requêtes HTTP** aux différents microservices via **le reverse proxy (Traefik).** Ces requêtes incluent généralement des méthodes standard telles que **GET, POST, PUT et DELETE**. L**'authentification** est assurée à l'aide de **tokens JWT**, qui sont transmis dans les en-têtes des requêtes.

Donc:

- le frontend envoie une requête de connexion au service d'authentification,
- après validation, un token JWT est retourné,
- ce token est ensuite utilisé pour accéder aux endpoints du service API,
- le service API peut, à son tour, interroger le service d'intelligence artificielle pour obtenir une prédiction.

Ce mode de communication garantit **une interaction directe et rapide** entre les composants, ce qui est essentiel pour **les fonctionnalités en temps réel** comme la traduction des gestes.

1.5.2 Communication asynchrone : RabbitMQ

En complément de la **communication synchrone**, une communication **asynchrone** a été mise en place à l'aide de RabbitMQ. Ce mécanisme permet de gérer efficacement les tâches longues ou non critiques sans bloquer le traitement principal.

Le principe repose sur **un système de file de messages** :

- les services producteurs publient des messages dans une file,
- les services consommateurs **(workers)** récupèrent ces messages et exécutent les tâches correspondantes.

Dans ce projet, RabbitMQ est utilisé pour :

- l'enregistrement de l'historique des traductions,
- le traitement différé de certaines données,
- la gestion des logs et des notifications.

Cette approche permet **d'améliorer les performances globales** du système en **réduisant le temps de réponse** perçu par l'utilisateur, tout en assurant une meilleure gestion de la charge.

La combinaison de la communication **synchrone et asynchrone** permet de tirer parti des avantages de chaque approche. **Les API REST** assurent des **échanges rapides et directs**, tandis que **RabbitMQ** permet de **gérer efficacement les tâches** en **arrière-plan**. Cette hybridation contribue à la performance, à la robustesse et à la scalabilité du système.

1.6 Infrastructure du système

L'architecture microservices mise en place repose sur plusieurs **composants d'infrastructure essentiels**, permettant **d'assurer la communication,** la **découverte des services, la gestion du trafic** ainsi que **la persistance des données**. Ces éléments **garantissent la robustesse**, **la scalabilité** et **la disponibilité** du système.

1.6.1 RabbitMQ : système de messagerie

RabbitMQ est utilisé comme **broker de messages** pour implémenter la **communication asynchrone** entre **les microservices**. Il permet de **découpler les composants** du système en introduisant **une file d'attente** pour les **tâches non critiques** ou **coûteuses** en temps de calcul.

Le fonctionnement repose sur **un modèle producteur/consommateur** :

- les services producteurs envoient des messages dans une file,
- les services consommateurs (workers) récupèrent ces messages et exécutent les tâches correspondantes.

Dans ce projet, **RabbitMQ** est principalement utilisé pour :

- l'enregistrement de l'historique des traductions,
- le traitement différé des données,
- la gestion des logs et des notifications.

L'utilisation de RabbitMQ permet **d'améliorer les performances globales** en **évitant le blocage des requêtes principales** et en assurant **une meilleure répartition** de la charge.

1.6.2 Consul : service de découverte

Consul est un outil de **service discovery** permettant aux différents microservices de s'**enregistrer** et de se **découvrir dynamiquement**. Dans **une architecture distribuée, les adresses des services** peuvent **changer**, ce qui rend nécessaire l'utilisation d'un mécanisme de **localisation dynamique**.
Chaque **microservice s'enregistre** auprès de **Consul** avec **ses informations** (**nom, adresse, port**). Les autres services peuvent ensuite **interroger Consul** pour **localiser** les services disponibles.

Les principales fonctionnalités de Consul sont :

- l'enregistrement automatique des services,
- la découverte dynamique des instances,
- la vérification de l'état des services (health check).

Grâce à Consul, le système devient **plus flexible et résilient**, car les services peuvent être **ajoutés, supprimés ou redémarrés** sans perturber l'ensemble de l'application.

1.6.3 Traefik : reverse proxy et load balancer

Traefik est utilisé comme **point d'entrée unique du système**. Il agit à la fois comme **reverse proxy** et comme **répartiteur de charge** (load balancer). **Toutes les**

requêtes externes passent par **Traefik avant** d'être **redirigées vers les microservices** appropriés.

Ses principales fonctions incluent :

- le routage des requêtes vers les services concernés,
- la gestion du **load balancing** entre plusieurs instances d'un même service,
- l'intégration avec Consul pour la découverte **dynamique,**
- la gestion des certificats SSL (optionnel).

Dans ce projet, **Traefik permet de simplifier l'accès aux services en offrant une interface unique**, tout en optimisant la distribution du trafic et en améliorant la disponibilité du système.

1. 6.4 SQLife : système de gestion de base de données

Dans le cadre de ce projet, le système de gestion de base de données utilisé est **SQLite**, qui est **intégré par défaut** avec **le framework Django**. Ce choix a permis de simplifier **le développement et le déploiement** de l'application, notamment dans un environnement de test ou de prototypage.

SQLite est une base de données relationnelle légère qui ne nécessite pas de serveur dédié. Elle stocke les données dans un fichier local, ce qui facilite son utilisation et sa configuration.

Les principales caractéristiques de **SQLite** incluent :

- une installation simple sans configuration complexe,
- une intégration native avec Django,
- une légèreté adaptée aux projets de taille moyenne,
- une utilisation efficace pour les phases de développement et de test.

Dans ce projet, **SQLit**e est utilisée pour :

- la gestion des comptes utilisateurs,
- l'enregistrement des résultats de traduction,
- la conservation de l'historique des interactions.

Cependant, bien que SQLite soit adaptée au développement, elle présente des limitations en termes de performance et de gestion des accès concurrents dans des environnements distribués.

1.7 Sécurité

La sécurité constitue **un élément essentiel** dans une architecture microservices, notamment pour **protéger les données** des utilisateurs et **contrôler l'accès** aux

différentes fonctionnalités. Dans ce projet, une approche basée sur **l'authentification centralisée** et **l'utilisation de tokens JWT** a été adoptée.

1.7.1 Authentification basée sur JWT

Le mécanisme principal de **sécurisation** repose sur l'utilisation des **JSON Web Tokens (JWT)**. **Ce type de token** permet d'**authentifier** les utilisateurs de manière **stateless,** c'est-à-dire sans conserver de session côté serveur.

Lorsqu'un utilisateur se connecte :

- ses identifiants sont envoyés au service d'authentification,
- après validation, un token JWT est généré,
- ce token est ensuite utilisé pour accéder aux autres services.

Le token est transmis dans l'en-tête des requêtes HTTP, généralement sous la forme :

Authorization: Bearer <token>

Les avantages de l'utilisation **des JWT** sont :

- une réduction de la charge serveur (pas de session à stocker),
- une meilleure scalabilité,
- une compatibilité avec les architectures distribuées.

1.7.2 Authentification centralisée

L'architecture adoptée repose sur un service d'**authentification**, responsable de la gestion des accès et des permissions. Tous les utilisateurs doivent s'authentifier via ce service avant d'accéder aux autres microservices.

Ce modèle présente plusieurs avantages :

- une gestion unifiée de la sécurité,
- une simplification du contrôle d'accès,
- une cohérence dans la validation des requêtes.

Les autres services (**API, AI,** etc.) ne gèrent pas directement l'authentification, mais **vérifient la validité du token JWT** avant de traiter les requêtes.

1.8 Déploiement du système

Le déploiement constitue une étape essentielle dans la mise en œuvre d'une architecture microservices, car il permet de simuler un environnement distribué réel et de valider le bon fonctionnement des différents composants.

1.8.1 Conteneurisation avec Docker

Dans ce projet, **la conteneurisation** a été réalisée à l'aide de **Docker.** Chaque microservice est encapsulé dans un conteneur **indépendant,** incluant ses dépendances et sa configuration. Cette approche permet d'assurer la portabilité et la reproductibilité de l'application sur différents environnements.

Les principaux avantages de Docker dans ce contexte sont :

- l'isolation des services,
- la facilité de déploiement,
- la cohérence entre les environnements de développement et de production,
- la simplification de la gestion des dépendances.

Un fichier Docker Compose est utilisé pour **orchestrer** le **lancement simultané** de tous **les services**, incluant **le frontend**, **les microservices backend**, ainsi que les **composants d'infrastructure** (RabbitMQ, Consul, Traefik).

1.8.2 Déploiement multi-serveurs

Afin de respecter les exigences du projet, chaque **microservice est déployé sur un serveur distinct**, simulant ainsi une architecture distribuée. Cette séparation permet d'améliorer la scalabilité et la tolérance aux pannes.

La répartition des services est présentée dans le tableau suivant :

Service	Machine
Auth Service	**Serveur 1**
API Service	**Serveur 2**
Frontend	**Serveur 3**
RabbitMQ	**Serveur 4**
Consul	**Serveur 5**
Traefik	**Serveur frontal**

Cette organisation permet de distribuer la charge de travail et d'assurer une meilleure disponibilité des services.

L'utilisation de **Docker combinée à une architecture multi-serveurs** permet de construire un système flexible, évolutif et proche des conditions réelles de production.

1.9 Guide de déploiement

Le déploiement du système suit une série d'étapes ordonnées afin d'assurer le bon démarrage et la communication entre les différents services.

Étapes de déploiement

a) **Lancement du broker RabbitMQ** :Ce composant doit être démarré en premier afin de permettre la gestion des communications asynchrones.

b) **Démarrage de Consul (service discovery)** : Les microservices s'enregistrent auprès de Consul, ce qui permet leur découverte dynamique.

c) **Déploiement des microservices** : Les services (Auth, API, AI, Worker) sont lancés et se connectent aux composants d'infrastructure.

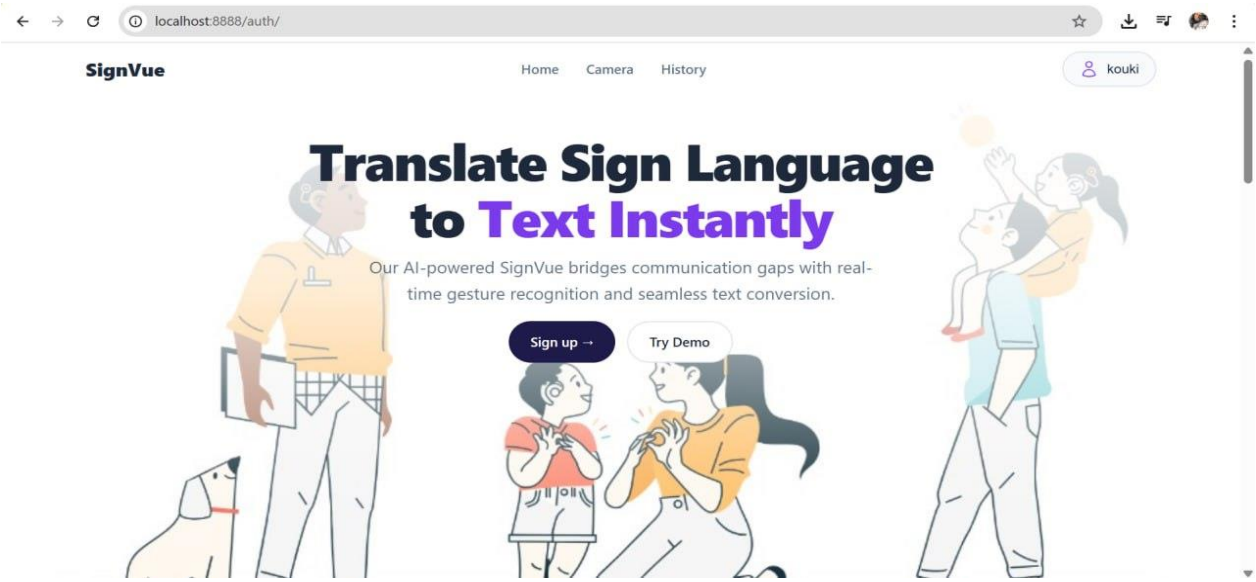
d) **Configuration et lancement de Traefik** : Traefik est configuré comme point d'entrée du système, assurant le routage des requêtes vers les services appropriés.

e) **Test du système** : Vérification du bon fonctionnement des communications REST et RabbitMQ, ainsi que de l'accès aux différents endpoints.

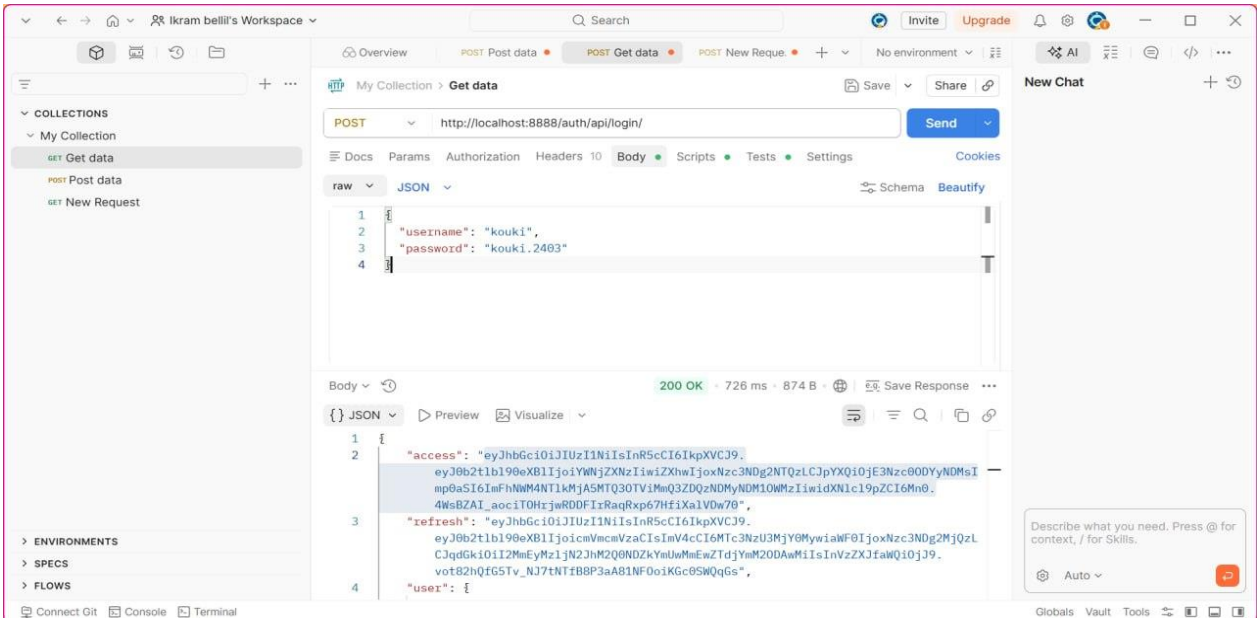
1.10 validation du système

Cette section présente les captures d'écran réalisées lors des tests du système. Elles permettent de valider le bon fonctionnement des différents microservices .

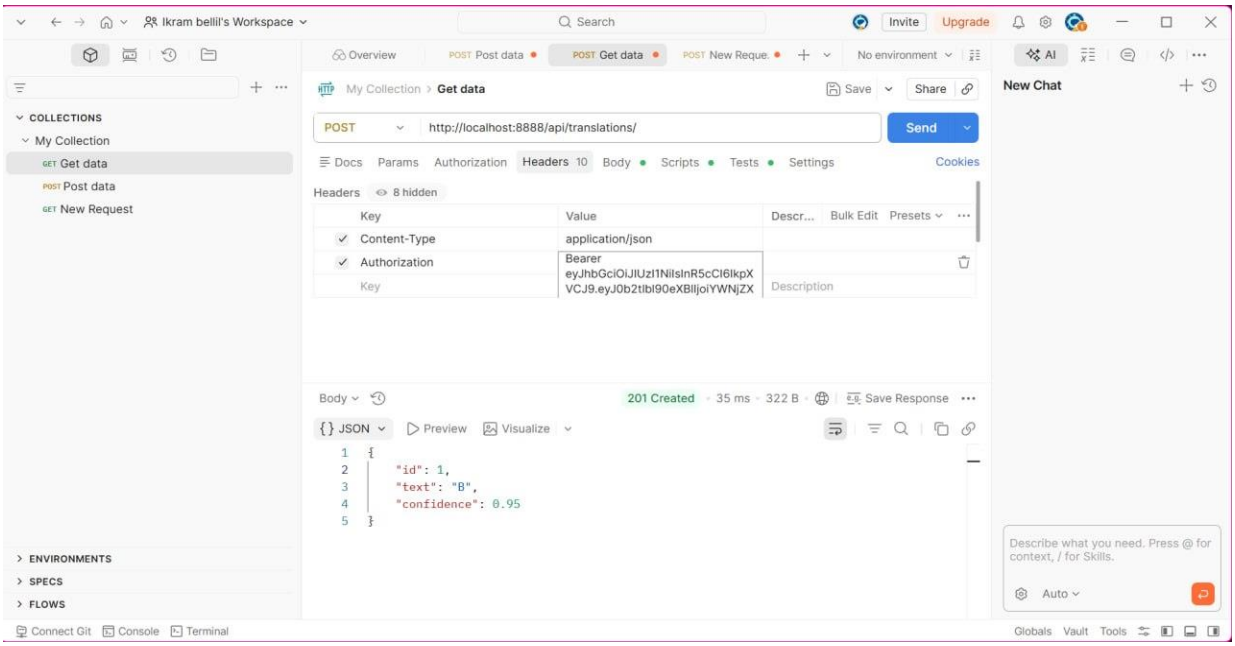
Test d'accès au site (Frontend) : page d'accueil



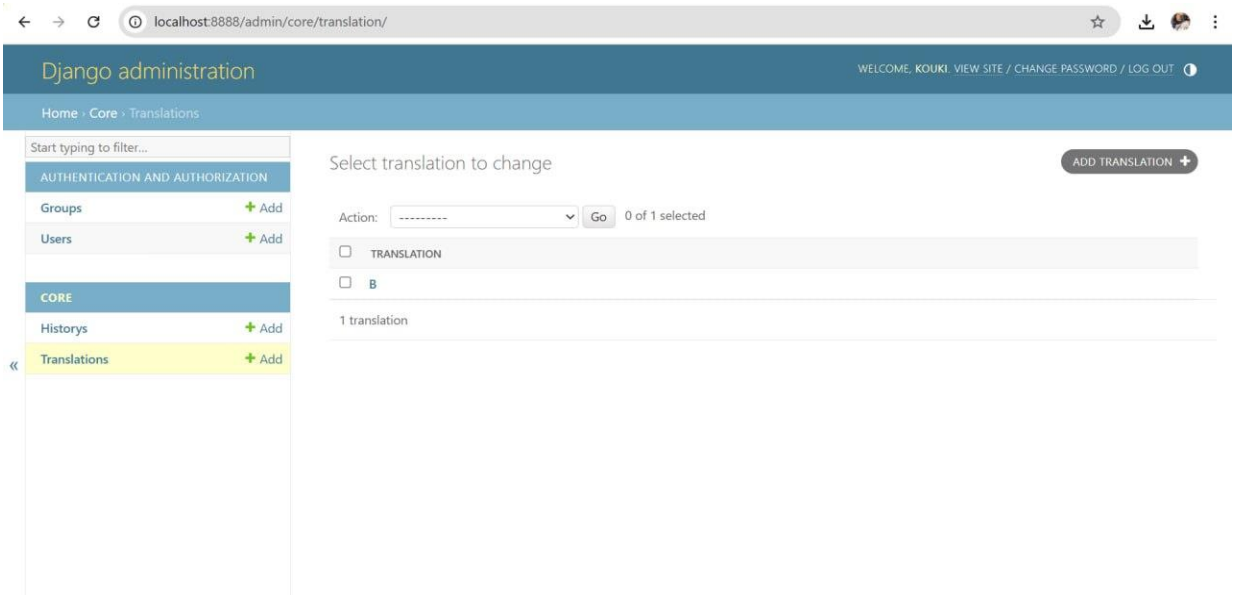
Test d'authentification JWT : login réussi + token généré



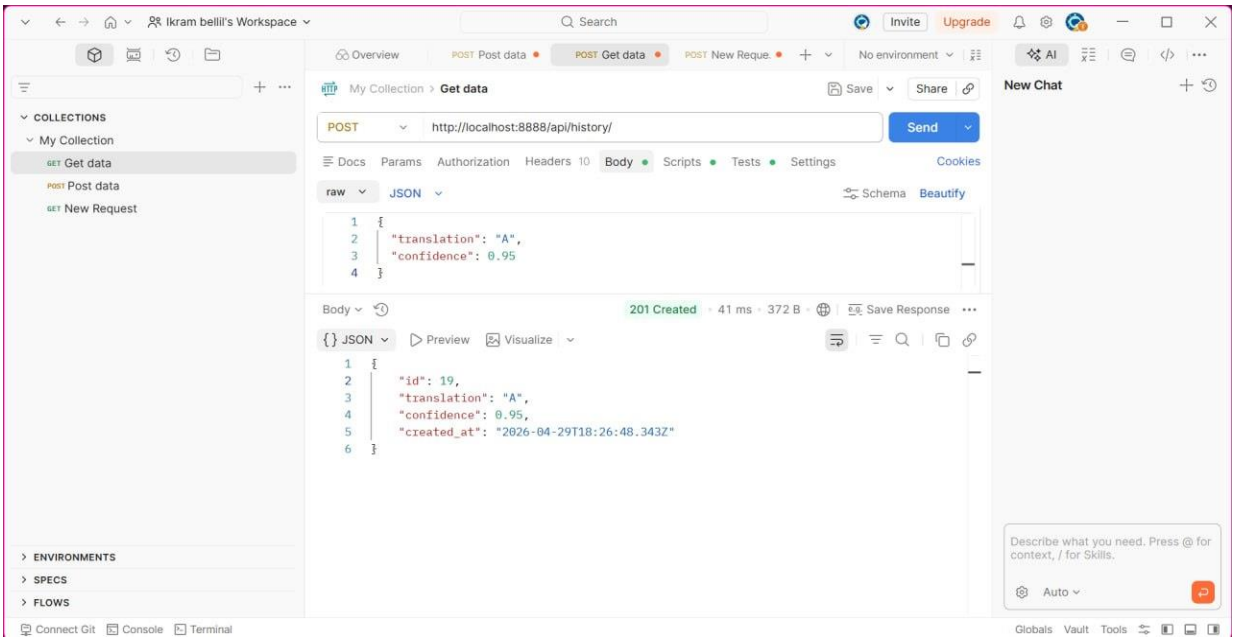
Test ajout translation : envoi donnée vers API (translation ajoutée + données enregistrées)



Test table translations (BDD) : Vérification base de données + données stockées correctement



Test création history (automatique) : généralement, history se remplit automatiquement quand tu fais une action (ex: translation ou prediction)



Test affichage history : récupération données → affichage des actions utilisateur

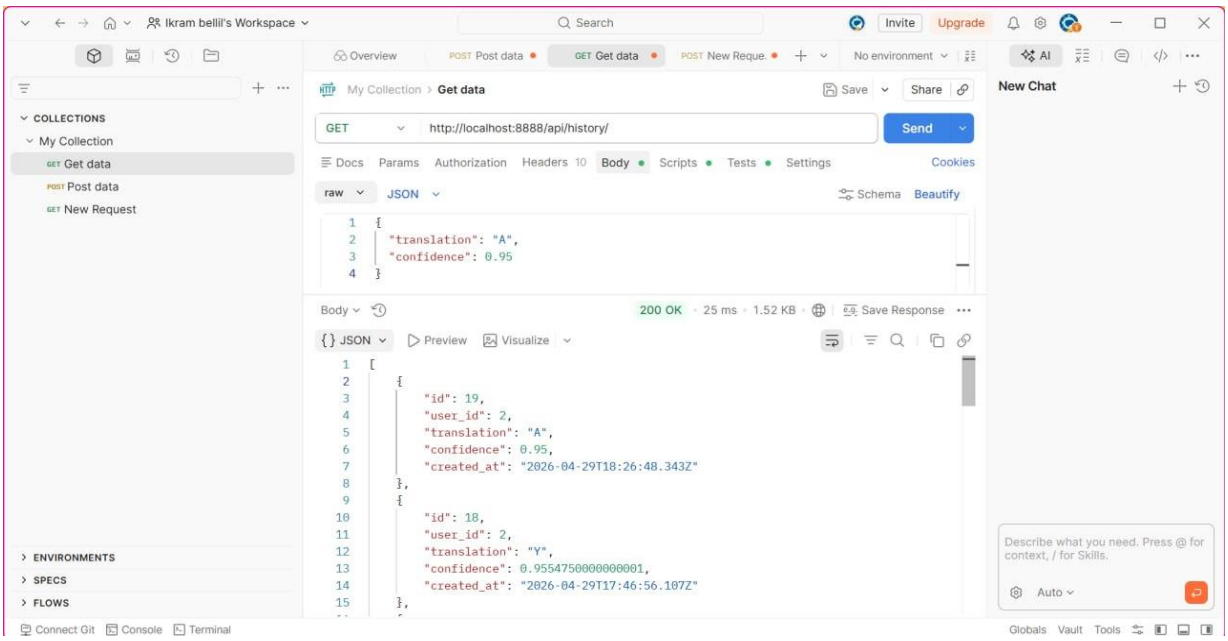
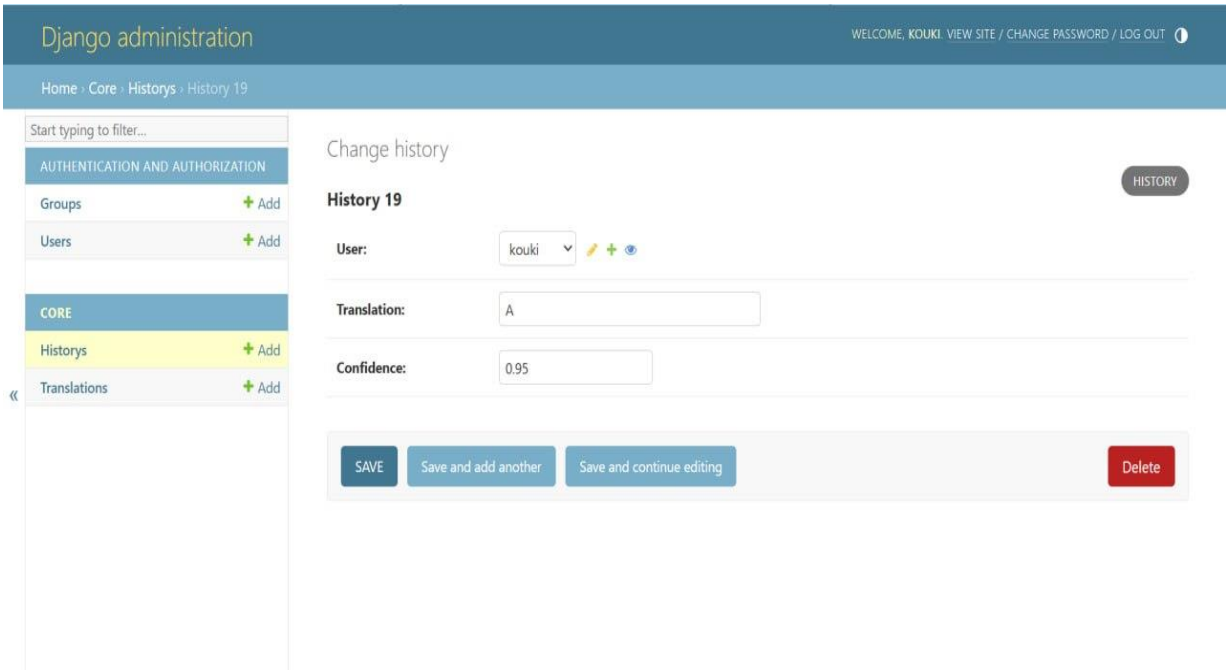
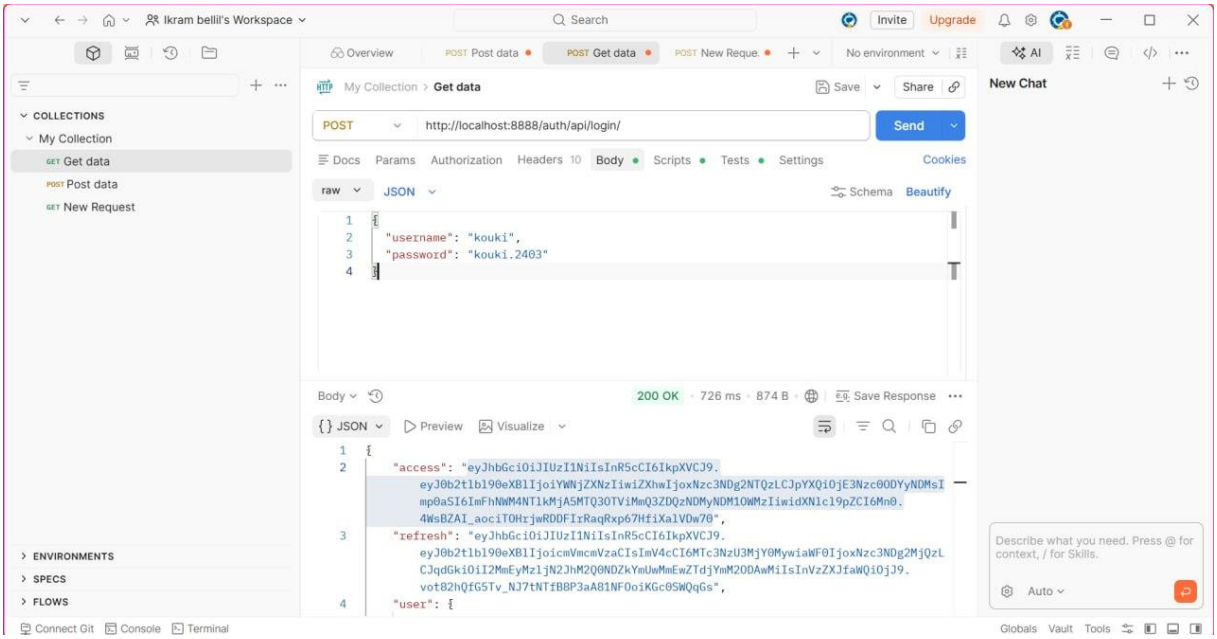


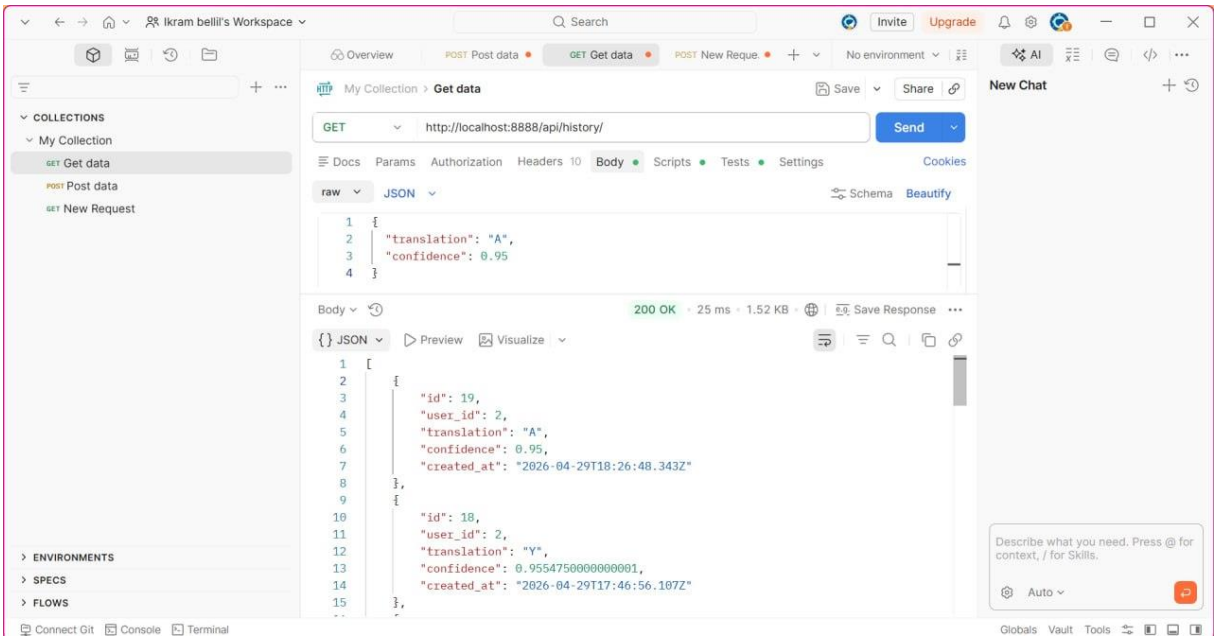
Table History se remplit automatiquement



TEST JWT AUTHENTICATION : L'utilisateur envoie ses identifiants (username / password). Si les informations sont correctes authentification réussie +génération d'un **token JWT**

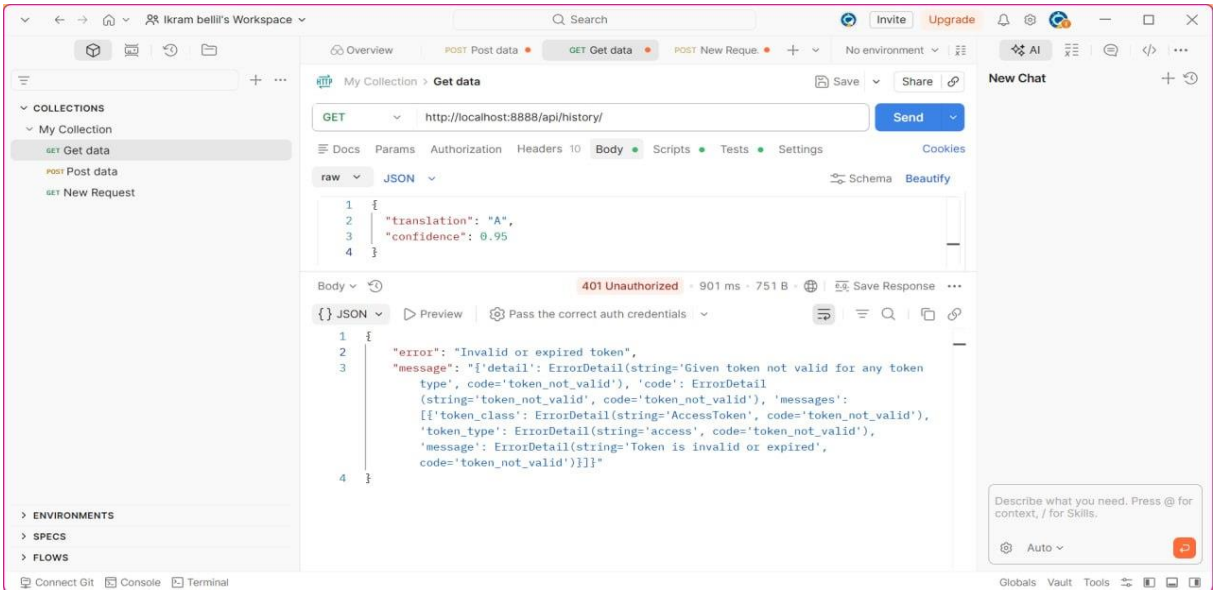


Test d'accès autorisé (token valide) : L'utilisateur envoie une requête avec un *token JWT valide.* → accès autorisé à l'API + retour des données demandées



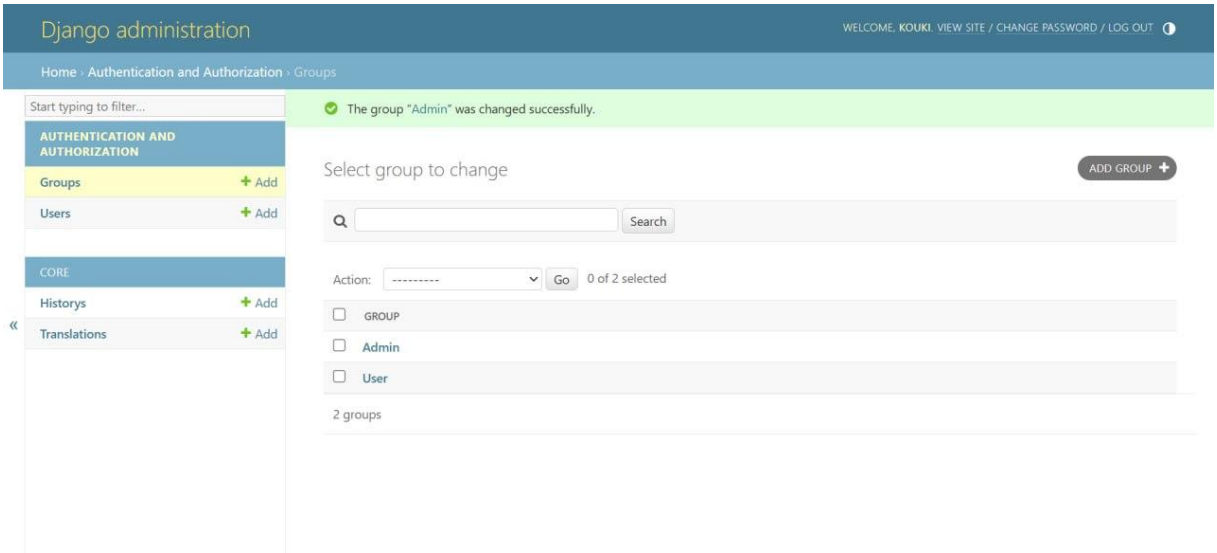
Test sans token (sécurité) : Requête envoyée sans token. → accès refusé +erreur 401 Unauthorized

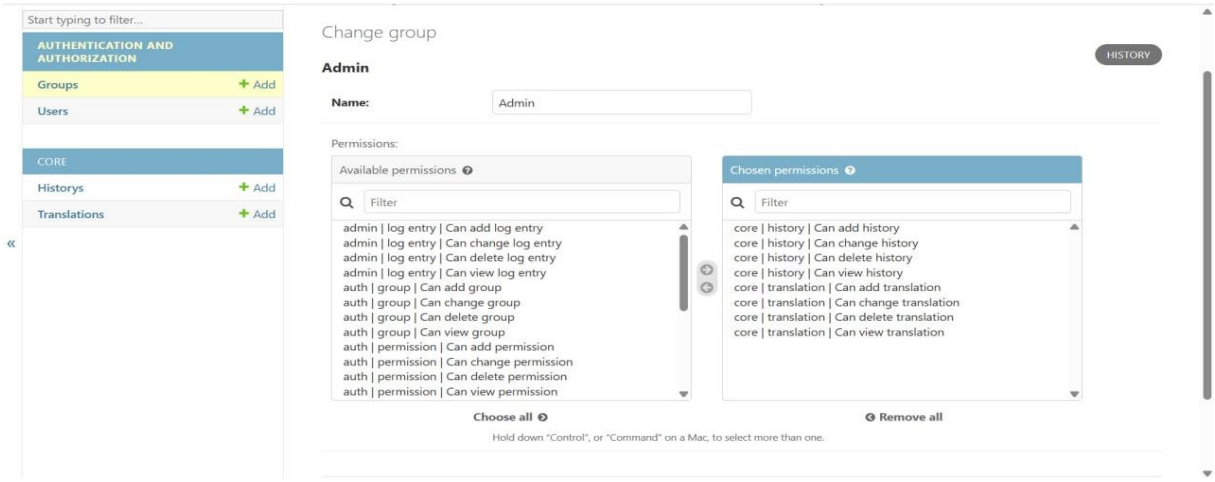
Nous avons intégré JWT pour sécuriser l'API. Chaque utilisateur doit s'authentifier pour obtenir un token. Sans token valide, l'accès est refusé avec une erreur 401 Unauthorized

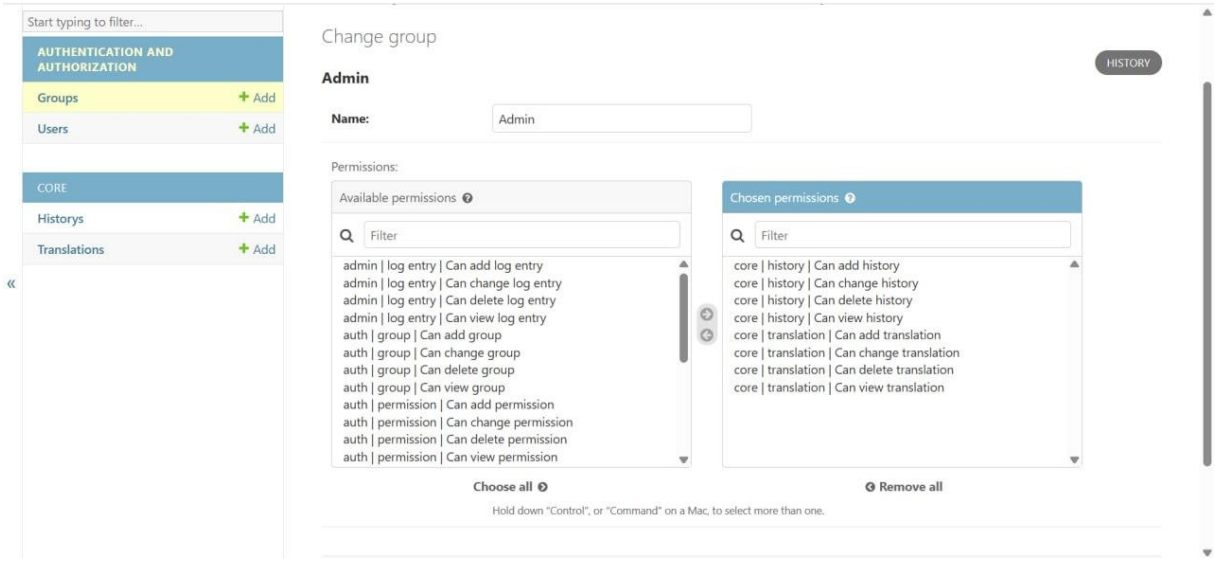


Nous avons implémenté un système **RBAC** avec deux groupes **: Admin et User**. L'Admin possède toutes les permissions CRUD (add, view, change, delete), tandis que le User possède uniquement add et view.

```
Python 3.11.15 (main, Apr 22 2026, 02:06:58) [GCC 14.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> from django.contrib.auth.models import Group
>>>
>>> admin = Group.objects.get(name="Admin")
>>>
>>> print("=== ADMIN PERMISSIONS ===")
=== ADMIN PERMISSIONS ===
>>> for perm in admin.permissions.all():
...     print(perm.codename)
...
add_history
change_history
delete_history
view_history
add_translation
change_translation
delete_translation
view_translation
>>> from django.contrib.auth.models import Group
>>>
>>> user = Group.objects.get(name="User")
>>>
>>> print("=== USER PERMISSIONS ===")
=== USER PERMISSIONS ===
>>> for perm in user.permissions.all():
...     print(perm.codename)
...
add_history
view_history
add_translation
view_translation
```

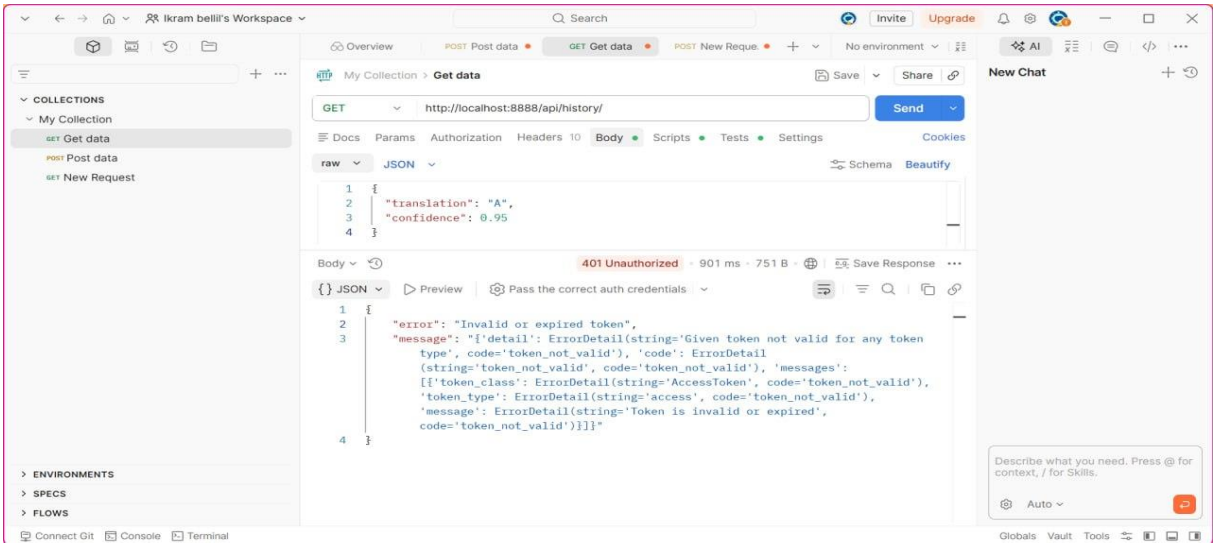




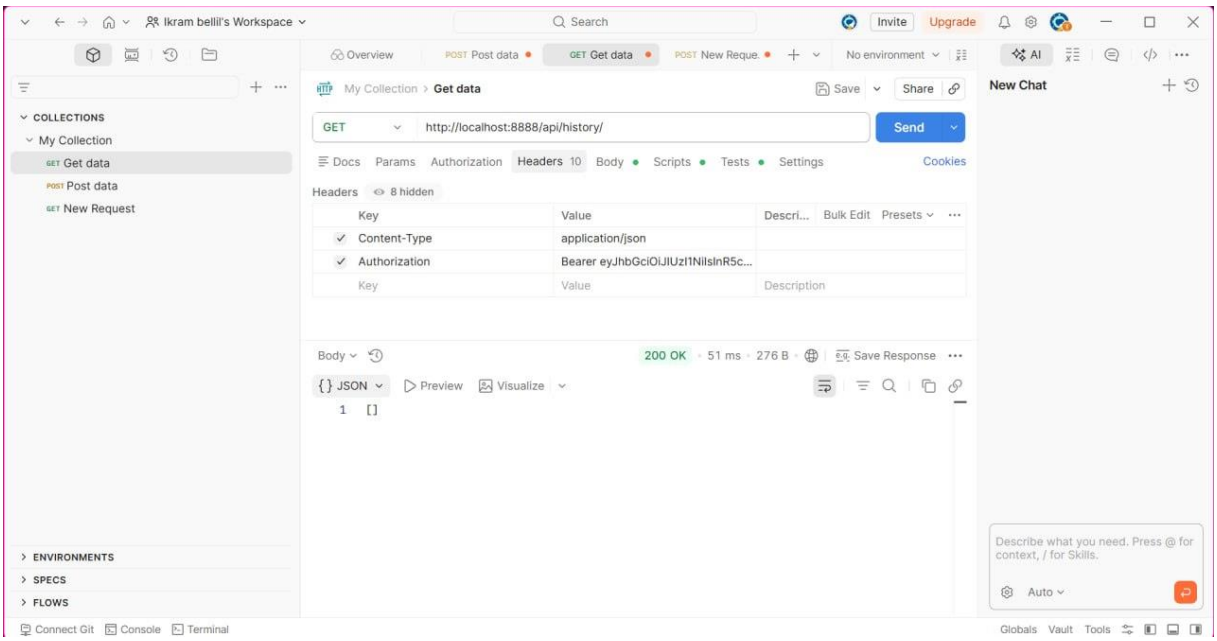


Nous avons implémenté une séparation des données au niveau des APIs. L'administrateur peut accéder à l'ensemble de l'historique du système, tandis que chaque utilisateur ne peut voir que ses propres données via JWT et les permissions Django

ADMIN → VOIR TOUT L'HISTORIQUE



Utilisateur voit uniquement ses données



1.11 Conclusion de la partie

L'architecture microservices proposée dans ce projet permet de structurer le système de manière modulaire, scalable et maintenable. **Grâce à la séparation des responsabilités**, chaque composant peut évoluer indépendamment, tout en garantissant une communication efficace via des mécanismes synchrones et asynchrones.

L'intégration d'outils tels que Docker, RabbitMQ, Consul et Traefik renforce la robustesse du système et permet de simuler un environnement distribué réaliste. Cette approche répond aux exigences des applications modernes nécessitant flexibilité, performance et disponibilité.

Cependant, bien que cette architecture assure la gestion efficace des flux de données et des interactions entre services, la qualité globale du système dépend fortement de la précision et des performances du modèle d'intelligence artificielle utilisé pour la reconnaissance des gestes.

Ainsi, la deuxième partie de ce travail sera consacrée à l'étude et à la mise en œuvre du modèle de Deep Learning, qui constitue le cœur fonctionnel du système de traduction de la langue des signes**.**

PARTIE 2 : Apprentissage profond

2.1 Introduction de la partie

La vision par ordinateur (Computer Vision) est un domaine de **l'intelligence artificielle** qui permet aux machines d'analyser et de comprendre **des informations visuelles** issues d'**images** ou de **vidéos.** Grâce aux avancées récentes en **Deep Learning**, ces systèmes sont désormais capables **d'identifier des objets,** de **reconnaître des visages** et **d'interpréter des gestes** avec **une grande précision**.

Dans le contexte de la communication humaine, **la vision par ordinateur** joue un rôle essentiel dans **la reconnaissance de la langue des signes.** En effet, les personnes sourdes ou malentendantes utilisent cette langue pour communiquer, mais celle-ci reste difficilement compréhensible pour les personnes ne la maîtrisant pas.

Dans ce projet, nous **avons développé un modèle d'apprentissage profond** basé sur **l'alphabet de la langue des signes américaine (ASL),** permettant la **reconnaissance** automatique **des lettres à partir d'images de gestes de la main**. Ce système vise à transformer **des gestes complexes en texte**, contribuant ainsi à réduire les barrières de communication.

2.2 Intégration de l'IA en microservice

Les modèles de **Deep Learning** sont généralement **complexes et gourmands** en ressources. Leur intégration directe dans une application monolithique pose plusieurs problèmes :

- forte consommation mémoire
- dépendance au matériel (GPU)
- difficulté de mise à jour du modèle
- augmentation du temps de réponse

Pour pallier ces limitations, le modèle est **encapsulé dans un microservice indépendant.**

2.2.1 Principe de l'AIaaS

Le concept d'Intelligence Artificielle en tant que Service (AIaaS) consiste à exposer les fonctionnalités du modèle via une API.

L'application cliente envoie une image au service. Le service IA traite cette image et une prédiction est retournée sous forme JSON

2.2.2 Implémentation du microservice

Le microservice a été développé à l'aide du framework Django. Il expose **deux endpoints** principaux :

Endpoint de vérification : « *GET /health* »
Ce point permet de vérifier :

- que le service est actif
- que le modèle est correctement chargé en mémoire

Endpoint de prédiction : « *POST /predict* »
Entrée :

- image encodée en Base64 ou envoyée sous forme de fichier

Sortie :

- classe prédite
- probabilité associée

2.2.3 Pipeline de traitement

Lorsqu'une requête est envoyée, les étapes suivantes sont exécutées :

Étape 1 : Lecture de la requête
La requête JSON est analysée afin d'extraire l'image.

```
data = json.loads(request.body)
image_base64 = data.get("image")
```

Étape 2 : Décodage
L'image encodée en Base64 (La fonction ***decode_base64_image() :***) est convertie en tableau de pixels puis transformée en image RGB exploitable.

Étape 3 : Prétraitement
L'image est transformée afin de correspondre aux exigences du modèle (taille, normalisation). Le modèle **MobileNetV2** étant entraîné sur des images 224x224

```
Resize(224, 224) → ToTensor() →
Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225])
```

Étape 4 : Inférence
Le modèle pré-entraîné est utilisé pour générer une prédiction.

```
torch.no_grad() :
outputs = current_model(image_tensor)
probabilities = torch.softmax(outputs, dim=1)
predicted_class = torch.argmax(probabilities, dim=1).item()
```

Le modèle produit un vecteur de 29 valeurs, chaque valeur représentant une probabilité pour l'une des classes.

Étape 5 : Post-traitement
La sortie du modèle est convertie en lettre correspondante. À l'aide du tableau ASL_CLASSES,

ASL_CLASSES = ['A', 'B', ..., 'Z', 'delete', 'nothing', 'space']
predicted_letter = ASL_CLASSES[predicted_class]

Étape 6 : Réponse
Le résultat est renvoyé sous forme JSON.

{"prediction": "A", "confidence": 0.97, "class_index": 0 }

2.2.4 Optimisation et performance

Plusieurs optimisations ont été mises en place :

- Chargement du modèle une seule fois au démarrage
- Utilisation de torch.no_grad() pour réduire la consommation mémoire
- Gestion des exceptions pour assurer la robustesse du service
- Possibilité de déploiement sur plusieurs instances pour gérer la montée en charge

2.3 Dataset

Le modèle est entraîné à partir du dataset ASL Alphabet Dataset disponible sur Kaggle.

2.3.1 Description générale

Le dataset utilisé dans ce projet est le ASL Alphabet Dataset, disponible publiquement sur la plateforme Kaggle .Ce dataset contient environ 87 000 images réparties sur 29 classes (29 gestes) :

- 26 lettres de l'alphabet (A à Z)
- 3 classes supplémentaires : space, delete, nothing
- images en couleur (RGB)
- fond généralement uniforme
- variations de position de la main
- bonne qualité visuelle

Chaque image représente une main réalisant un geste spécifique.

2.3.2 Limites du dataset

Malgré sa taille importante, le dataset présente certaines limites :

- faible diversité de fonds
- conditions d'éclairage peu variées
- manque de diversité morphologique (formes de mains)

Ces limitations peuvent affecter la généralisation du modèle en conditions réelles.

geste correspondant à la lettre A (différentes positions de la main)

2.4 Prétraitement des données

Le prétraitement constitue une étape essentielle pour garantir de bonnes performances du modèle.

2.4.1 Redimensionnemen

Toutes les images sont redimensionnées en **224 × 224 pixels**.
Ce choix est dicté par :

- les contraintes du modèle MobileNetV2 : ***transforms.Resize((224,224))***
- la nécessité d'uniformiser les données d'entrée

2.4.2 Normalisation

Les valeurs des pixels sont normalisées avec une moyenne de [0.485, 0.456, 0.406] et un écart-type de [0.229, 0.224, 0.225].selon les statistiques du dataset ImageNet sur lequel le modèle a été pré-entraîné

mean = [0.485, 0.456, 0.406] std = [0.229, 0.224, 0.225]

Objectifs :
une convergence rapide sans uniformité. Certaines valeurs de pixels peuvent être élevées, jusqu'à 255, ce qui entraîne une inflation du gradient pendant l'entraînement. La sélection du genre permet au modèle de se concentrer uniquement sur la forme de la main plutôt que sur l'arrière-plan et les autres éléments, améliorant ainsi l'efficacité du transfert et de l'apprentissage.

2.4.3 Data Augmentation

Afin d'améliorer la robustesse du modèle, plusieurs transformations aléatoires sont appliquées :

- rotations
- symétries
- variations de luminosité

Cela permet de simuler des conditions réelles et d'augmenter artificiellement la taille du dataset.

2.5 Modèle : MobileNetV2

2.5.1 Présentation

MobileNetV2 est un réseau de neurones convolutif conçu pour les environnements à ressources limitées.

2.5.2 Justification du choix

Ce modèle a été choisi pour :

- sa légèreté
- sa rapidité d'exécution
- sa compatibilité avec le temps réel
- ses performances élevées sur des tâches de classification

2.5.3 Architecture

Le modèle repose sur plusieurs types de couches :
Convolution : Extraction automatique des caractéristiques visuelles.

Pooling :Les couches de pooling permettent de réduire la taille des cartes de caractéristiques.

- réduction du nombre de paramètres
- diminution du temps de calcul

- réduction du risque de surapprentissage

Dense :Les couches entièrement connectées (Dense) permettent de combiner les caractéristiques extraites afin de réaliser la classification finale. Dans notre modèle, nous utilisons :

- une couche Dense de 256 neurones
- une couche de sortie correspondant aux 29 classes

Dropout : Régularisation pour éviter le surapprentissage.
Nous avons intégré la technologie Dropout, avec des taux de **0,3** et **0,2** , dans le code. Ensemble, **ces taux désactivent certains neurones** pendant l'entraînement afin de **prévenir spécifiquement** la sur-composition.

Softmax : Transformation des sorties en probabilités.
Il s'agit de la dernière étape du modèle. C'est une fonction mathématique qui convertit les nombres produits par le modèle en probabilités comprises entre 0 et 1, la somme totale étant égale à 100 %.

2.6 Entraînement du modèle

Le modèle a été entraîné avec les paramètres suivants :

- **Epochs** : 30 itérations complètes sur l'ensemble du jeu de données.
- **Batch size** : 32 images par lot, offrant un bon équilibre entre stabilité du gradient et utilisation de la mémoire GPU.
- **Optimizer** : Algorithme Adam est utilisé avec un taux d'apprentissage initial de 0.001
- **Learning rate** : 0.001
- **Loss function** : CrossEntropyLoss. Elle est adaptée aux problèmes de classification multi-classes.

2.6.1 Processus d’apprentissage

Lors de chaque époque, le modèle suit les étapes suivantes :

- Parcours de l’ensemble des données d’entraînement
- Calcul de la fonction de perte (loss)
- Ajustement des poids du modèle à l’aide de l’algorithme de rétropropagation (backpropagation)

Ce processus est répété plusieurs fois afin d’optimiser progressivement les performances du modèle.

2.6.2 Suivi des performances

Afin d'évaluer correctement le comportement du modèle durant l'apprentissage, plusieurs indicateurs suivants ont été utilisés :

- **Accuracy :** mesure le pourcentage de prédictions correctes
- **Loss** : mesure l'erreur entre les prédictions du modèle et les valeurs réelles
- **Courbes d'apprentissage :** permettent de visualiser l'évolution de l'accuracy et de la loss au fil des époques
- **Matrice de confusion :** permet d'analyser les erreurs de classification entre les différentes classes

Nous n'avons pas seulement suivi le taux de réussite global ; nous avons adopté une stratégie d'évaluation complète afin de garantir la stabilité et la robustesse du modèle.

En analysant les courbes de précision (accuracy) et de perte (loss), nous avons pu contrôler le processus d'apprentissage et vérifier que le modèle n'était pas sujet à un surapprentissage (overfitting).

La matrice de confusion nous a également fourni des informations précieuses sur les classes visuellement similaires, permettant d'évaluer la capacité du modèle à distinguer des gestes proches en langue des signes.

Après 30 époques d'entraînement, le modèle a montré une amélioration progressive de ses performances, indiquant une convergence correcte.

2.7 Évaluation du modèle

L'évaluation permet de mesurer la capacité du modèle à généraliser sur de nouvelles données.

- **Accuracy** : Proportion de bonnes prédictions.
- **Loss** : Erreur entre prédictions et valeurs réelles.

Une diminution de la loss accompagnée d'une augmentation de l'accuracy indique un apprentissage efficace.

2.8 Résultats

Le modèle effectue une classification parmi **29 classes** (A–Z, space, delete, nothing).
La sortie du modèle correspond à une distribution de probabilités sur ces classes.
Exemple : A : 95 % / E : 3 % / S : 2 %

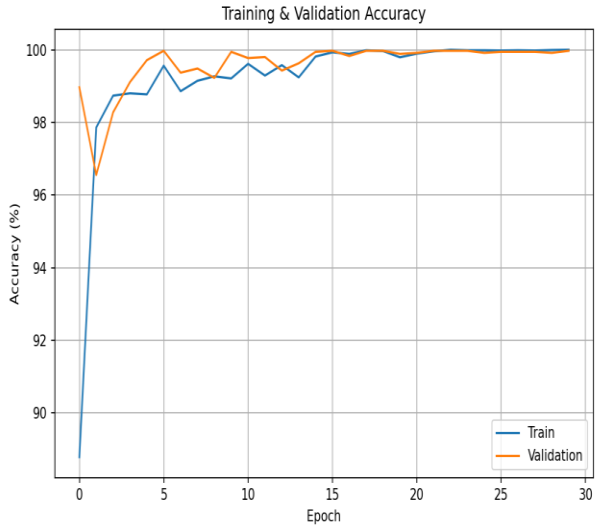
La classe ayant la probabilité la plus élevée est sélectionnée comme prédiction finale.
Les résultats obtenus montrent :

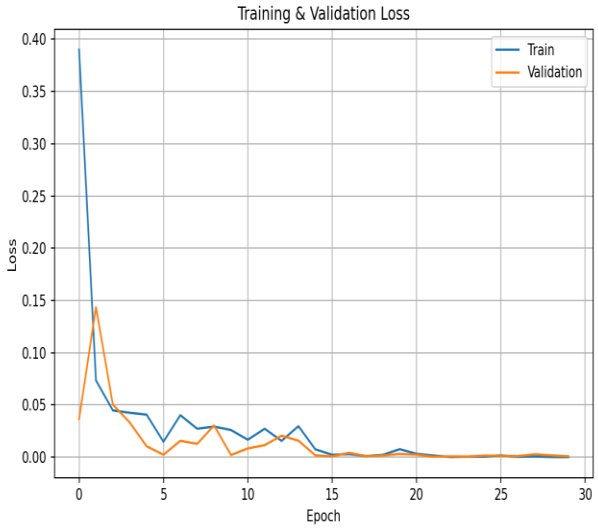
- une précision élevée
- une bonne stabilité du modèle
- une faible différence entre les performances d'entraînement et de validation

Ces observations indiquent une bonne capacité de généralisation.

Résultats finaux de l'entraînement :

Train-Accrucy	val-Accrucy	Train-Loss	Val-loss
99.97%	**99,94%**	**0.0002%**	**0,0032%**





Ces résultats montrent que le modèle atteint un niveau de précision très élevé aussi bien sur les données d'entraînement que sur les données de validation.
Nous constatons que la différence entre la précision d'entraînement et la précision de validation est très faible, ce qui indique une excellente capacité de généralisation du modèle.
Par ailleurs, la valeur très faible de la fonction de perte (loss) confirme que le modèle converge correctement et parvient à minimiser efficacement l'erreur de prédiction.

2.8 Discussion

Malgré les performances élevées obtenues, une analyse approfondie permet d'identifier certains points importants concernant le comportement du modèle.

2.9.1 Overfitting

Le surapprentissage (overfitting) se produit lorsque le modèle apprend trop précisément les données d'entraînement , ce qui peut dégrader ses performances sur de nouvelles données. Pour limiter ce phénomène, plusieurs techniques ont été utilisées :

- data augmentation
- dropout
- Utilisation d'un jeu de validation

De plus, l'analyse des courbes d'apprentissage ainsi que la faible différence entre les performances d'entraînement et de validation confirment que le modèle ne présente pas de surapprentissage significatif.

2.9.2 Limites du dataset

Malgré la taille importante du dataset, certaines limites doivent être soulignées :

- Fond souvent uniforme
- Faible variation des conditions d'éclairage
- Diversité limitée des formes de main

Ces contraintes peuvent impacter les performances du modèle en conditions réelles, où l'environnement est plus complexe.

2.9.3 Analyse des erreurs

L'utilisation de la matrice de confusion a permis d'identifier certaines confusions entre classes visuellement similaires.
En particulier, certaines lettres proches en termes de position des doigts peuvent être mal classifiées. Cela montre que le modèle peut rencontrer des difficultés à distinguer des gestes très similaires.

2.9.4 Intégration système

L'intégration du modèle dans une architecture microservices constitue une étape essentielle pour transformer le modèle en un service opérationnel.
Le modèle est encapsulé dans une API REST exposant un endpoint /predict, permettant de :

- recevoir une image depuis l'interface utilisateur
- effectuer une prédiction en temps réel
- retourner le résultat sous forme de texte

Cette approche présente plusieurs avantages :

- modularité du système
- indépendance du service IA
- facilité de mise à jour du modèle
- possibilité de déploiement à grande échelle

2.10 Conclusion

Dans cette partie, on a créé un modèle d'apprentissage profond qui peut reconnaître les gestes de la langue des signes américaine (ASL) à partir d'images.

Les résultats montrent que les CNN, c'est-à-dire les réseaux de neurones convolutifs, sont vraiment efficaces pour reconnaître des images et des gestes. Le modèle a prouvé qu'il pouvait 60% identifier les lettres de l'alphabet en langue des signes.

Cependant, plusieurs améliorations peuvent être envisagées afin d'augmenter les performances et la robustesse du système :

- utilisation des séquences vidéo au lieu d'images statiques
- reconnaissance de mots ou de phrases complètes
- enrichissement du dataset avec des conditions plus réalistes
- déploiement sur des plateformes mobiles ou web

À terme, ce projet constitue une première étape vers la conception d'un système intelligent capable de faciliter la communication entre les personnes sourdes et entendantes grâce aux technologies d'intelligence artificielle et de vision par ordinateur.

V. CONCLUSION GÉNÉRALE

Dans ce projet, nous avons développé un système intelligent de reconnaissance de la langue des signes en combinant une architecture microservices et des techniques de Deep Learning.

Dans la première partie, nous avons conçu une architecture microservices permettant de structurer l'application en services indépendants, assurant modularité, flexibilité et scalabilité. L'utilisation de composants tels que le service d'authentification, la communication asynchrone, le service de découverte et le reverse proxy a permis de construire un système robuste et bien organisé.

Dans la seconde partie, nous avons développé un modèle de Deep Learning basé sur MobileNetV2 pour la reconnaissance des gestes de la langue des signes à partir d'images. Les résultats obtenus montrent une bonne précision et une capacité satisfaisante de généralisation. L'intégration du modèle sous forme de microservice a permis de séparer efficacement la logique d'intelligence artificielle du reste du système.

Cependant, certaines limites subsistent, notamment liées au dataset utilisé et à la reconnaissance limitée aux lettres isolées. Des améliorations futures pourraient inclure l'utilisation de vidéos en temps réel, la reconnaissance de mots complets ainsi que l'enrichissement des données d'apprentissage.

En conclusion, ce projet démontre l'efficacité de la combinaison entre microservices et intelligence artificielle pour développer des solutions innovantes, tout en ouvrant la voie à des améliorations futures visant à faciliter la communication entre personnes sourdes et entendantes.

Références

ASL Alphabet Dataset **"ASL Alphabet Dataset (Kaggle)"**
National College of Ireland. **Guttula, H. (2024).** ***Deep Learning Approaches to Real-Time Sign Language Recognition and Multilingual Translation.***
Articles scientifiques sur MobileNetV2 et la reconnaissance de la langue des signes